CERTIK

Cetus - CLMM

Security Assessment

CertiK Assessed on Oct 16th, 2026



CertiK Assessed on Oct 16th, 2026

Cetus - CLMM

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
CLMM	Sui (SUI)	Manual Review, Static Analysis

LANGUAGE	TIMELINE
Move	Preliminary comments published on 04/04/2025
	Final report published on 10/16/2026

Vulnerability Summary



19	14	0	5	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

1	Centralization	1 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
0	Medium		Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
4	Minor	4 Resolved	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
14	Informational	10 Resolved, 4 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | CETUS - CLMM

Audit Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Overview

External Dependencies

Privileged Roles

Upgradeability

Findings

CEA-02 : Centralization Related Risks And Upgradability

CEA-04 : Cross-Module Publisher Misuse In `pool` And `position` Modules

CEA-05 : Potential Division By Zero In `get_next_sqrt_price_a_up` Function

CEA-21 : Incorrect Snapshot Logic In `apply_liquidity_cut` May Block Position Closure

CEA-22 : Add/Remove/Repay Liquidity Sequence Allows Flashloan Like Operation With No Cost

CEA-03 : Potential Pitfall With `in_allowed_list` And `in_denied_list` Validation

CEA-06 : Potential Inconsistent CoinType Order In `register_permission_pair_internal` Function

CEA-08 : Discussion On Return `false` With `is_right_order`

CEA-09 : Potential Invalid Initialization Price Validation

CEA-10 : Missing Validation On Tick Spacing

CEA-12 : Discussion On PARTNER_RATE

CEA-13 : Edge Case In `get_liquidity_by_amount` Inactive Liquidity At Lower Tick

CEA-14 : Potential DoS On This Function When The List Of Member Getting Long

CEA-15 : Discussion Of Underflow Handling

CEA-17 : Potential Inconsistent Liquidity After The Recovery

CEA-18 : Discussion On Reward And Position Update

CEA-19 : Potential Blocker In Emergency Workflow

CEA-20 : Discussion On `emergency_restore_pool_state`

CEC-01 : Liquidity Rounding Can Strand A Material Portion Of Fixed-Coin Deposits

Optimizations

CEA-11 : Comment Error

CEA-16 : Unused Remove Function in Position Snapshot

Appendix

Disclaimer

CODEBASE | CETUS - CLMM

Repository

cetus_clmm

Commit

390ee5b030de44a782dc0fc6f23061611a92a64b

4b243d4bb07aaed89dd44d8f1991cb9a54a5f586

bda1995f7f704548c2c3954fa3014dc38c3f4c26

0df3adad7d3e41c24e64e418d38173c48221fe32

29decc06fe3246af77a3d54166954f919fdd48c2

ffd7345d8bb3d11023e8c119a9d056400626b284

Audit Scope

The files in scope are listed in the appendix.

APPROACH & METHODS | CETUS - CLMM

This report has been prepared for Cetus to discover issues and vulnerabilities in the source code of the Cetus - CLMM project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

REVIEW NOTES | CETUS - CLMM

Overview

The **Cetus** is a pioneer DEX and concentrated liquidity protocol built on the Sui blockchain.

External Dependencies

The project is developed using the Move language and running on the top of the Sui blockchain. The vulnerability and the updates of the language/Sui framework may affect the project as a whole. As the Sui network is rapidly evolving, to avoid any potential compatibility issues and take advantage of new features and improvements, the client should upgrade the Sui framework to the most recent version. Additionally, staying informed about any upcoming updates or changes to the language or framework can help ensure the project remains secure and compatible.

Dependency of the **Cetus**:

```
Sui = { git = "https://github.com/MystenLabs/sui.git", subdir = "crates/sui-framework/packages/sui-framework", rev = "mainnet-v1.42.2", override = true }
MoveSTL = { git = "https://github.com/CetusProtocol/move-stl.git", subdir = "sui", rev = "mainnet-v1.23.1", override = true }
IntegerMate = { git = "https://github.com/CetusProtocol/integer-mate.git", subdir = "sui", rev = "mainnet-v1.23.1", override = true }
```

The above dependencies are not within the current audit scope and serve as a black box. Modules/Contracts within the module are assumed to be valid and non-vulnerable actors in this audit and implement proper logic to collaborate with the current project and other modules.

Privileged Roles

To set up the project correctly and ensure that the project functions properly, owners of the following objects are able to use privileged functions, more details in the **Centralization** findings.

The advantage of the privileged role in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to serve the community best. It is also worthy of note the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the key pairs of privileged accounts are compromised, the project could have devastating consequences.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Furthermore, any plan to invoke the aforementioned functions should also be considered to move to the execution queue of the `Timelock` contract.

Upgradeability

Developers working with the Sui blockchain have the ability to upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and publisher's key store should be handled with caution to

prevent any unexpected loss. Additionally, it is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

Reference:

- Sui Package Upgrades
- Custom Policies

FINDINGS | CETUS - CLMM



19	0	1	0	0	4	14
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for Cetus to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 19 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
CEA-02	**Centralization Related Risks And Upgradability**	**Centralization**	**Centralization**	**Acknowledged**
CEA-04	Cross-Module Publisher Misuse In `pool` And `position` Modules	Access Control	Minor	Resolved
CEA-05	Potential Division By Zero In `get_next_sqrt_price_a_up` Function	Volatile Code	Minor	Resolved
CEA-21	Incorrect Snapshot Logic In `apply_liquidity_cut` May Block Position Closure	Design Issue	Minor	Resolved
CEA-22	Add/Remove/Repay Liquidity Sequence Allows Flashloan Like Operation With No Cost	Logical Issue	Minor	Resolved
CEA-03	Potential Pitfall With `in_allowed_list` And `in_denied_list` Validation	Volatile Code	Informational	Acknowledged
CEA-06	Potential Inconsistent CoinType Order In `register_permission_pair_internal` Function	Inconsistency	Informational	Resolved
CEA-08	Discussion On Return `false` With `is_right_order`	Coding Issue	Informational	Resolved
CEA-09	Potential Invalid Initialization Price Validation	Volatile Code	Informational	Resolved
CEA-10	Missing Validation On Tick Spacing	Volatile Code	Informational	Resolved

ID	Title	Category	Severity	Status
CEA-12	Discussion On PARTNER_RATE	Volatile Code	Informational	Acknowledged
CEA-13	Edge Case In `get_liquidity_by_amount` Inactive Liquidity At Lower Tick	Volatile Code	Informational	Resolved
CEA-14	Potential DoS On This Function When The List Of Member Getting Long	Volatile Code	Informational	Acknowledged
CEA-15	Discussion Of Underflow Handling	Logical Issue	Informational	Resolved
CEA-17	Potential Inconsistent Liquidity After The Recovery	Coding Issue	Informational	Resolved
CEA-18	Discussion On Reward And Position Update	Logical Issue	Informational	Resolved
CEA-19	Potential Blocker In Emergency Workflow	Volatile Code	Informational	Resolved
CEA-20	Discussion On `emergency_restore_pool_state`	Logical Issue	Informational	Resolved
CEC-01	Liquidity Rounding Can Strand A Material Portion Of Fixed-Coin Deposits	Volatile Code	Informational	Acknowledged

CEA-02 | Centralization Related Risks And Upgradability

Category	Severity	Location	Status
Centralization	Centralization	sources/config.move (cetus_clmm): 188, 210, 236, 257, 285, 300, 315, 329, 431; sources/factory.move (cetus_clmm): 259, 285, 300, 315, 321, 336, 366, 415, 564; sources/partner.move (cetus_clmm): 128, 223, 244; sources/pool.move (cetus_clmm): 1116, 1149, 1179, 1217, 1237, 1278, 1296; sources/rewarder.move (cetus_clmm): 347	Acknowledged

Description

config.move

AdminCap

In the module **config**, the role **AdminCap** has authority over the functions:

- set_roles()
- add_role()
- remove_role()
- remove_member()
- update_package_version()

Any compromise to the holder for **AdminCap** may allow a hacker to take advantage of this authority, manage the roles, and update the package version.

ACL_POOL_MANAGER

In the module **config**, the role **ACL_POOL_MANAGER** has authority over the functions:

- update_protocol_fee_rate()

Any compromise to the **ACL_POOL_MANAGER** account may allow a hacker to take advantage of this authority and update the protocol fee.

ACL_FEE_TIER_MANAGER

In the module **config**, the role **ACL_FEE_TIER_MANAGER** has authority over the functions:

- add_fee_tier()
- delete_fee_tier()

- update_fee_tier()

Any compromise to the **ACL_FEE_TIER_MANAGER** account may allow a hacker to take advantage of this authority and manage the fee tier.

ACL_EMERGENCY_PAUSE

In the module **config**, the role **ACL_EMERGENCY_PAUSE** has authority over the functions:

- emergency_pause()

Any compromise to the **ACL_EMERGENCY_PAUSE** account may allow a hacker to take advantage of this authority and disable functionalities via version.

ACL_EMERGENCY_UNPAUSE

In the module **config**, the role **ACL_EMERGENCY_UNPAUSE** has authority over the functions(added in V15):

- emergency_unpause()

Any compromise to the **ACL_EMERGENCY_UNPAUSE** account may allow a hacker to take advantage of this authority and enable functionalities via version. And note due to the sui upgrade mechanism, v14 can still using ACL_EMERGENCY_PAUSE role to unlock the PAUSE.

---

factory.move

ACL_POOL_MANAGER

In the module **factory**, the role **ACL_POOL_MANAGER** has authority over the functions:

- init_manager_and_whitelist()
- add_allowed_list()
- remove_allowed_list()
- add_denied_list()
- remove_denied_list()
- add_allowed_pair_config()
- remove_allowed_pair_config()
- mint_pool_creation_cap_by_admin()
- create_pool()

Any compromise to the **ACL_POOL_MANAGER** account may allow a hacker to take advantage of this authority, create the pool, and manage the configuration of the pools.

TreasuryCap

In the module **factory**, the holder of a `TreasuryCap` can create `PoolCreationCap` via `mint_pool_creation_cap`.

PoolCreationCap

In the module **factory**, the `PoolCreationCap` can:

- register_permission_pair()
- unregister_permission_pair()

---

partner.move

ACL_PARTNER_MANAGER

In the module **partner**, the role **ACL_PARTNER_MANAGER** has authority over the functions:

- create_partner()
- update_ref_fee_rate()
- update_time_range()

Any compromise to the **ACL_PARTNER_MANAGER** account may allow a hacker to take advantage of this authority, create a partner, and manage the configuration of the partner.

PartnerCap

In the module **partner**, the holder of `PartnerCap` has the authority to collect the reference fee via `claim_ref_fee`.

---

pool_creator.move

The holder of `PoolCreationCap` maintains the ability `create_pool_v2_with_creation_cap` to create a new pool.

---

pool.move

AdminCap

In the module **pool**, the owner of object `AdminCap` has the authority to create `ProtocolFeeCollectCap`.

ProtocolFeeCollectCap

In the module **pool**, the owner of object `ProtocolFeeCollectCap` accompany with `ACL_CLAIM_PROTOCOL_FEE` has the authority to collect protocol fee via `collect_protocol_fee_with_cap`.

ACL_POOL_MANAGER

In the module **pool**, the role **ACL_POOL_MANAGER** has authority over the functions:

- update_position_url()
- update_fee_rate()
- pause() (deprecated)
- unpause() (deprecated)

- init_position_snapshot()
- apply_liquidity_cut() (deprecated)
- governance_fund_injection() (deprecated)
- **governance_fund_withdrawal()** (deprecated)
- emergency_remove_malicious_position() (deprecated)
- emergency_restore_pool_state() (deprecated)
- set_pool_status()

Any compromise to an **ACL_POOL_MANAGER** member may allow an attacker to update the per-pool fee rate and position URL, or selectively disable and enable pool operations through `set_pool_status()`. During the emergency-restore state, the role can also initialize the position snapshot used by the recovery workflow.

It is worth noting that the functionalities, such as `governance_fund_withdrawal`, are intended to be used for the recovery; if any compromise on the corresponding authority may cause unlimited loss, and the team should be careful about such role management.

The following function has been set to deprecated with commit 0df3adad7d3e41c24e64e418d38173c48221fe32:

- apply_liquidity_cut()
- governance_fund_injection()
- governance_fund_withdrawal()
- emergency_remove_malicious_position()
- emergency_restore_pool_state()

ACL_REWARDER_MANAGER

In the module **pool**, the role **ACL_REWARDER_MANAGER** has authority over the functions:

- initialize_rewarder()
- update_emission()

Any compromise to the **ACL_REWARDER_MANAGER** account may allow a hacker to take advantage of this authority and manage the configuration of the reward.

ACL_CLAIM_PROTOCOL_FEE

In the module **pool**, the role **ACL_CLAIM_PROTOCOL_FEE** has authority over the functions:

- collect_protocol_fee()

Any compromise to the **ACL_CLAIM_PROTOCOL_FEE** account may allow a hacker to take advantage of this authority and collect the protocol fee.

Publisher

In the module **pool**, the publisher would set the pool display.

Any compromise to the **publisher** account may allow a hacker to affect the display feature.

---

position.move

Publisher

In the module **position**, the publisher would set the pool display.

Any compromise to the **publisher** account may allow a hacker to affect the display feature.

---

rewarder.move

AdminCap

In the module **rewarder**, the role **AdminCap** has authority over the functions:

- emergent_withdraw()

Any compromise to the **AdminCap** account may allow a hacker to take advantage of this authority and withdraw rewards in an emergency.

In addition, developers working with the Sui blockchain can upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and deployer's key store should be handled carefully to prevent any unexpected losses. It is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

More information can be found:

- Sui Package Upgrades
- Third-Party Package upgrades

In the updated pull request, the following functions have been deprecated:

- `apply_liquidity_cut()`
- `governance_fund_injection()`
- `governance_fund_withdrawal()`
- `emergency_remove_malicious_position()`
- `emergency_restore_pool_state()`

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Cetus Team, 04/18/2025]: We have already migrated all administrative privileges to be managed by a multi-signature address, which effectively reduces the risk of single points of failure and increases the overall security of the system.

Regarding the evidence submission issue, we noticed that the form provided does not have an option for SUI chain. We will follow up with the team to ensure that this option is added, allowing us to submit mitigation evidence related to the SUI chain.

[CertiK, 04/18/2025]: The finding can be updated after the details are provided. It is suggested to implement the aforementioned methods to avoid centralized failure. Also, it strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

CEA-04 Cross-Module Publisher Misuse In `pool` And `position` Modules

Category	Severity	Location	Status
Access Control	Minor	sources/pool.move (cetus_clmm): 364; sources/position.move (cetus_clmm): 137	Resolved

Description

In the `pool` and `position` modules, the `publisher` can be improperly utilized to set display information across modules. Although the `Display` module includes internal checks to verify that the publisher originates from the current package, it does not confirm if the publisher is from the corresponding module. This oversight allows the publisher from the `pool` module to invoke the `set_display` function of the `position` module, leading to potential unauthorized modifications.

Reference: https://move-book.com/programmability/publisher.html#usage

Recommendation

It is suggested to implement additional validation within the `pool` and `position` modules to ensure that the `publisher` is strictly used within its corresponding module.

Alleviation

[Cetus Team, 04/16/2025]: The team heeded the advice and resolved the issue by heeding the advice in commit d00f3268b50ac8acc0c8675feddf5041a8cbb4be.

CEA-05 Potential Division By Zero In `get_next_sqrt_price_a_up` Function

Category	Severity	Location	Status
Volatile Code	Minor	sources/math/clmm_math.move (cetus_clmm): 169~171	Resolved

Description

In the `get_next_sqrt_price_a_up` function, there is a potential division by zero error when the condition `liquidity_shl_64 >= product` is required, but `liquidity_shl_64` equals `product`. The formula `(math_u256::div_round(numerator, (liquidity_shl_64 - product), true) as u128)` will fail due to a zero denominator, which can lead to unexpected errors.

```
...
if (liquidity_shl_64 < product) {
    abort ESUBTRACTION_UNDERFLOW
};
(math_u256::div_round(numberator, (liquidity_shl_64 - product), true) as u128)
...
```

Recommendation

Modify the condition from `<` to `<=` to prevent the division by zero error.

Alleviation

[Cetus Team, 04/16/2025]: The team heeded the advice and resolved the issue by heeding the advice in commit d00f3268b50ac8acc0c8675feddf5041a8cbb4be.

CEA-21 Incorrect Snapshot Logic In `apply_liquidity_cut` May Block Position Closure

Category	Severity	Location	Status
Design Issue	Minor	sources/pool.move (cetus_update): 829, 1700~1707	Resolved

Description

The `apply_liquidity_cut` function is designed to proportionally reduce the liquidity of specific positions following a data recovery event. As part of this process, each affected position is added to a snapshot.

However, the snapshot logic in the current implementation introduces a flaw:

```
position_snapshot::add(snapshot, position_id, cut_value, *position_info);

let (tick_lower, tick_upper) = position::info_tick_range(position_info);

let delta_liquidity = position_snapshot::calculate_remove_liquidity(snapshot,
position_info);
if (delta_liquidity == 0) {
    return;
};
```

Even when `delta_liquidity == 0`, the position is still added to the snapshot via `position_snapshot::add`. This becomes problematic because **any position that exists in the snapshot cannot be closed** due to the following invariant enforced when closing positions in the system:

```
assert!(!position_snapshot::contains(snapshot, position_id),
ECannotCloseAttackedPosition);
```

As a result, **positions with no actual liquidity reduction** are still locked in the system, preventing users from closing them. This behavior can lead to unnecessary user friction or stuck positions, especially for those unaffected by the cut in terms of actual liquidity.

Recommendation

It is suggested to modify the logic in `apply_liquidity_cut` to conditionally add the position to the snapshot **only if** `delta_liquidity > 0`. This ensures that positions which do not require any liquidity adjustment are not unintentionally locked.

Also, we would like to check with the team if the positions in the snapshot are intentionally cannot be closed.

Alleviation

[Cetus Team, 07/29/2025]: We'd like to clarify that the `apply_liquidity_cut` function is only called via an off-chain controlled recovery process. During this process, the off-chain logic **precomputes** the `cut_value` for each affected position. If the `cut_value` is zero, the off-chain logic **will skip calling** `apply_liquidity_cut` for that position entirely.

Therefore, in our intended usage, positions with zero liquidity reduction will never be added to the snapshot, and this locking behavior does not occur in practice.

Additionally, all snapshot-related recovery and restore functions including `apply_liquidity_cut` are being deprecated in the upcoming version, and we have already submitted a corresponding merge request to remove them.

https://github.com/CetusProtocol/cetus-clmm-sui/pull/38/files

[CertiK, 07/31/2025]: The finding is marked as resolved since the following function has been set to deprecated with commit 0df3adad7d3e41c24e64e418d38173c48221fe32:

- `apply_liquidity_cut()`
- `governance_fund_injection()`
- `governance_fund_withdrawal()`
- `emergency_remove_malicious_position()`
- `emergency_restore_pool_state()`

CEA-22 Add/Remove/Repay Liquidity Sequence Allows Flashloan Like Operation With No Cost

Category	Severity	Location	Status
Logical Issue	Minor		Resolved

Description

The add-liquidity flow mints liquidity and updates ticks immediately, returning an `AddLiquidityReceipt` with the owed `amount_a` and `amount_b` that must be repaid in the same transaction.

No guard is preventing a user from calling `remove_liquidity` on the same position before calling `repay_add_liquidity`. This permits sourcing the repayment entirely from the pool itself within the same transaction (flashloan-like behavior), i.e. add_liquidity -> remove_liquidity -> <do something> -> repay_add_liquidity.

As a result, a flashloan fee bypass could have happened.

Proof of Concept

The following test describes the scenario that the user could add_liquidity -> remove_liquidity -> repay_add_liquidity to perform a flashloan-like operation with(almost zero cost):

```
#[test]
public fun test_add_remove_then_repay_add_liquidity_same_tx() {
    let ctx = &mut tx_context::dummy();
    let (clock, admin_cap, config) = init_test(ctx);

    let mut pool = pool::new_for_test<CoinA, CoinB>(
        100,
        get_sqrt_price_at_tick(i32::zero()),
        2000,
        string::utf8(b""),
        0,
        &clock,
        ctx,
    );
    let (tick_lower, tick_upper) = (i32::neg_from(1000), i32::from(1000));

    // dummy add liquidity
    let mut position_nft_dummy = pool::open_position(
        &config,
        &mut pool,
        i32::as_u32(tick_lower),
        i32::as_u32(tick_upper),
        ctx,
    );

    let add_delta_liquidity = 1_000_000u128;
    let receipt = pool::add_liquidity(&config, &mut pool, &mut position_nft_dummy,
add_delta_liquidity, &clock);
    let (need_a, need_b) = pool::add_liquidity_pay_amount(&receipt);

    let balance_a = balance::create_for_testing<CoinA>(need_a);
    let balance_b = balance::create_for_testing<CoinB>(need_b);
    pool::repay_add_liquidity(&config, &mut pool, balance_a, balance_b, receipt);
    transfer::public_transfer(position_nft_dummy, TEST_ADDR);

    // dummy add liquidity finish


    let mut position_nft = pool::open_position(
        &config,
        &mut pool,
        i32::as_u32(tick_lower),
        i32::as_u32(tick_upper),
        ctx,
    );

    // Add liquidity by fixed liquidity amount to obtain receipt
    let add_delta_liquidity = 1_000_000u128;
```

```
    let receipt = pool::add_liquidity(&config, &mut pool, &mut position_nft,
add_delta_liquidity, &clock);
    let (need_a, need_b) = pool::add_liquidity_pay_amount(&receipt);
    std::debug::print(&need_a); // 48769
    std::debug::print(&need_b); // 48769

    // Immediately remove the same liquidity amount
    let (mut recv_a, mut recv_b) = pool::remove_liquidity<CoinA, CoinB>(
        &config,
        &mut pool,
        &mut position_nft,
        add_delta_liquidity,
        &clock,
    );



    // do anything with the flashloan

    std::debug::print(&balance::value(&recv_a)); // 48768
    std::debug::print(&balance::value(&recv_b)); // 48768

    let balance_a = balance::create_for_testing<CoinA>(1);
    let balance_b = balance::create_for_testing<CoinB>(1);
    let (balance_a, balance_b) = (balance::join<CoinA>(&mut recv_a, balance_a),
balance::join<CoinB>(&mut recv_b, balance_b));

    std::debug::print(&balance_a); // 48769
    std::debug::print(&balance_b); // 48769

    pool::repay_add_liquidity(&config, &mut pool, recv_a, recv_b, receipt);

    // Assertions: removed amounts equal receipt owed amounts; position liquidity
back to zero
    // assert!(balance::value(&recv_a) == need_a, 0);
    // assert!(balance::value(&recv_b) == need_b, 0);
    assert!(position::liquidity(&position_nft) == 0, 0);

    // // Clean up balances
    // recv_a.destroy_for_testing();
    // recv_b.destroy_for_testing();

    // Return objects
    transfer::public_transfer(position_nft, TEST_ADDR);
    transfer::public_share_object(pool);
    close_test(admin_cap, config, clock);
}
```

Recommendation

Recommend considering revising the logic to avoid removing liquidity between the add liquidity.

Alleviation

[Cetus, 09/17/2025]: The team resolved the issue in commit 7e97163647e2b396bd0a5f0384fb6f524413dd55 by adding a mechanism for liquidity operations.

CEA-03 Potential Pitfall With `in_allowed_list` And `in_denied_list` Validation

Category	Severity	Location	Status
Volatile Code	Informational	sources/factory.move (cetus_clmm): 223~238	Acknowledged

Description

The current implementation of `is_allowed_coin` in `factory.move` will perform validation on the given coin type based on the status inside `in_allowed_list` and `in_denied_list`.

However, if one coin type appears in both the allowed and denied lists, the coin will be treated as allowed.

```
is_allowed = is_allowed || !in_denied_list<Coin>(pools);
```

Recommendation

Recommend revising the validation to avoid the edge case.

Alleviation

[Cetus Team, 04/16/2025]: The team acknowledged this issue and confirmed this is intended design. If a coin appears in both lists, it is treated as allowed. The rationale is to prioritize the allow list, so explicitly approved assets won't be accidentally blocked

CEA-06 Potential Inconsistent CoinType Order In `register_permission_pair_internal` Function

Category	Severity	Location	Status
Inconsistency	Informational	sources/factory.move (cetus_clmm): 484~488, 497~498	Resolved

Description

In the `register_permission_pair_internal` function, there is a potential inconsistency between the input type parameters `CoinTypeA` and `CoinTypeB` and the data stored in `cap_to_pool_key`. The function is designed to automatically reorder the coins to ensure the base coin is greater than the quote coin when generating the pool key.

```
        let pool_key = if (is_right) {
            new_pool_key<CoinTypeA, CoinTypeB>(tick_spacing)
        }else {
            new_pool_key<CoinTypeB, CoinTypeA>(tick_spacing)
        };
```

However, the stored values for `coin_a` and `coin_b` may not align with `CoinTypeA` and `CoinTypeB` in the associated pool, leading to potential mismatches.

```
table::add(cap_keys, pool_key, PoolKey {
    coin_a: cap.coin_type,
    coin_b: quote_coin,
    tick_spacing,
});
```

Recommendation

We recommend ensuring that the order of `coin_a` and `coin_b` in the `PoolKey` structure aligns with the associated pool.

Alleviation

[Cetus Team, 04/16/2025]: The team resolved this issue in commit d00f3268b50ac8acc0c8675feddf5041a8cbb4be.

CEA-08 | Discussion On Return `false` With `is_right_order`

Category	Severity	Location	Status
Coding Issue	Informational	sources/factory.move (cetus_clmm): 844~846, 844~846	Resolved

Description

The `is_right_order<CoinTypeA, CoinTypeB>()` function in the factory module silently returns false when both coin types are identical, whereas the related `new_pool_key<CoinTypeA, CoinTypeB>()` function explicitly aborts with the `ESameCoinType` error in the same scenario.

We would like to check with the team to see if this is intended.

Recommendation

It's recommended to abort rather than return `false`, or to add a check that ensures the input tokens are not of the same coin type before calling this function.

Alleviation

[Cetus Team, 04/16/2025]: The team resolved this issue in commit d00f3268b50ac8acc0c8675feddf5041a8cbb4be.

CEA-09 | Potential Invalid Initialization Price Validation

Category	Severity	Location	Status
Volatile Code	Informational	sources/math/tick_math.move (cetus_clmm): 49	Resolved

Description

The following concerns related to the price initialization:

In the `tick_math::get_tick_at_sqrt_price` function, the use of the `<=` condition allows the pool to be initialized with a `sqrt_price` equal to `MAX_SQRT_PRICE_X64`.

Given that `max_tick` is intended to serve as a boundary that's never reached, permitting initialization at this extreme introduces the risk of running the pool at excessively high or unstable price levels.

Reference: https://github.com/Uniswap/v3-core/blob/main/contracts/libraries/TickMath.sol#L62

```
assert!(sqrt_price >= MIN_SQRT_PRICE_X64 && sqrt_price <= MAX_SQRT_PRICE_X64,
EINVALID_SQRTPRICE);
```

Additionally, the `create_pool_v2` function first calculates the square root price based on the input lower and upper tick indexes using the `get_sqrt_price_at_tick` function. This function allows tick indexes to reach the defined minimum and maximum bounds:

```
public fun get_sqrt_price_at_tick(tick: i32::I32): u128 {
    assert!(i32::gte(tick, min_tick()) && i32::lte(tick, max_tick()),
EINVALID_TICK);
    if (i32::is_neg(tick)) {
        get_sqrt_price_at_negative_tick(tick)
    } else {
        get_sqrt_price_at_positive_tick(tick)
    }
}
```

If the lower tick index is equal to `min_tick`, the resulting square root price will be `tick_math::min_sqrt_price()` (4295048016u128). Then the following check:

```
assert!(
    lower_sqrt_price <= initialize_price && upper_sqrt_price > initialize_price,
    EInitSqrtPriceNotBetweenLowerAndUpper
);
```

which means the input `initialize_price` is allowed to be equal to `lower_sqrt_price` (4295048016u128).

However, elsewhere in the `create_pool_v2_` function, the following check is enforced:

```
assert!(
    initialize_price > tick_math::min_sqrt_price() && initialize_price <
tick_math::max_sqrt_price(),
    EInvalidSqrtPrice
);
```

This introduces an inconsistency: one check allows `initialize_price == min_sqrt_price()`, while the other explicitly requires it to be greater than `min_sqrt_price()`.

Recommendation

We recommend unifying these conditions to prevent unintended reverts and ensure logical consistency across the pool initialization logic.

Alleviation

[Cetus Team, 04/16/2025]: The team resolved this issue in commit d00f3268b50ac8acc0c8675feddf5041a8cbb4be by setting a limitation on the initial price to keep consistency.

CEA-10 | Missing Validation On Tick Spacing

Category	Severity	Location	Status
Volatile Code	Informational	sources/config.move (cetus_clmm): 212	Resolved

Description

The fee and tick spacing can be inserted with `add_fee_tier`.

However, there is no validation for tick spacing. Consequently, the tick spacing value can be zero or extremely large, approaching the maximum or minimum tick values. This situation can restrict liquidity positioning and result in concentrated liquidity.

Reference: https://github.com/Uniswap/v3-core/blob/main/contracts/UniswapV3Factory.sol#L64

Recommendation

We would like to check with the team to see if the scenario has been considered.

Alleviation

[Cetus Team, 04/16/2025]: The team resolved this issue in commit d00f3268b50ac8acc0c8675feddf5041a8cbb4be.

CEA-12 | Discussion On PARTNER_RATE

Category	Severity	Location	Status
Volatile Code	Informational	sources/partner.move (cetus_clmm): 22	Acknowledged

Description

The following concerns have been raised regarding `PARTNER_RATE`:

1. `PARTNER_RATE_DENOMINATOR` is currently unused. Instead, all calculations rely on `PROTOCOL_FEE_DENOMINATOR`. This raises concerns about potential issues if `PROTOCOL_FEE_DENOMINATOR` and `PARTNER_RATE_DENOMINATOR` diverge in value in the future.

2. It appears possible for the referral fee (`ref fee`) to be set at 100% of the protocol fee. Is this behavior intended?

Recommendation

We would like to confirm if the current implementation align with the intended design.

Alleviation

[Cetus Team, 04/16/2025]: The team acknowledged the issue and stated that

1. To avoid introducing unnecessary complexity and potential errors, we have opted to keep this design as is, ensuring that the two denominators remain consistent. Any future divergence is highly unlikely, but we will monitor and update if necessary.
2. Yes, the design allows for the referral fee to be set at 100% of the protocol fee.

CEA-13 Edge Case In `get_liquidity_by_amount` Inactive Liquidity At Lower Tick

Category	Severity	Location	Status
Volatile Code	Informational	sources/math/clmm_math.move (cetus_clmm): 460~461	Resolved

Description

The `get_liquidity_by_amount` function calculates the liquidity to be minted based on the provided token amount. In the CLMM model, liquidity is only active within a specific tick range. When `current_tick_index >= upper_index` or `current_tick_index < lower_index`, there is **no active liquidity**—the position is entirely out of range.

- If `current_tick_index < lower_index`, only **token A** is required, and liquidity should be computed solely based on the amount of token A.
- If `current_tick_index >= upper_index`, only **token B** is needed, and liquidity should be calculated from the token B amount.

In the following `else if` branch:

```
amount_b = amount;
if (i32::gte(current_tick_index, upper_index)) {
    _liquidity = get_liquidity_from_b(lower_price, upper_price, amount, false);
} else if (i32::gte(current_tick_index, lower_index)) {
    _liquidity = get_liquidity_from_b(lower_price, current_sqrt_price, amount,
false);
    amount_a = get_delta_a(current_sqrt_price, upper_price, _liquidity, true);
} else {
    abort EINVALID_FIXED_TOKEN_TYPE;
}
```

This logic aims to handle the mid-range case where `lower_index <= current_tick_index < upper_index`. However, there is a **critical edge case**: when `current_tick_index == lower_index`, the tick is not yet active, and liquidity should not be initialized.

In fact, the function `get_liquidity_from_b` will **abort** when `current_tick_index == lower_index`, because it requires the tick to be strictly within the active range.

A similar concern also raised for `get_liquidity_from_amount`.

Recommendation

It is recommended to update the conditional check to avoid `current_tick_index == lower_index` when `is_fixed_a == false`.

Alleviation

[Cetus Team, 04/18/2025]: The team resolved the issue in commit 4b243d4bb07aaed89dd44d8f1991cb9a54a5f586.

CEA-14 Potential DoS On This Function When The List Of Member Getting Long

Category	Severity	Location	Status
Volatile Code	Informational	sources/acl.move (cetus_clmm): 80	Acknowledged

Description

The `get_members` function in the ACL module iterates through all members stored in a linked table and builds a vector containing all member information. In Sui Move, vectors have a size limitation (a known maximum size of ≤ 1000 items refer to the doc). If the number of members in the ACL grows very large, the function may hit this vector size limit, causing a transaction failure.

Recommendation

Recommend using dynamic field-backed collections to avoid unexpected result.

Alleviation

[Cetus Team, 04/16/2025]: The team acknowledged this issue and confirmed that it will monitor usage and optimize if necessary in the future.

CEA-15 | Discussion Of Underflow Handling

Category	Severity	Location	Status
Logical Issue	Informational	sources/position.move (cetus_clmm): 641, 644; sources/tick.move (cetus_clmm): 359~362	Resolved

Description

In the calculations for fees, rewards, and points inside of tick ranges, underflows are allowed by using `wrapping_sub`. An example of this can be found in the `get_rewards_in_range()` function, where an underflow might occur during the computation of `rewarder_inside`:

```
359             let rewarder_inside = math_u128::wrapping_sub(
360                 math_u128::wrapping_sub(growth_global, reward_growth_below),
361                 rewarder_growths_above
362             );
```

While these underflows are an intentional part of the calculation process, the auditing team seeks confirmation from the Cetus team that these underflows have been accounted for and managed outside of the tick and position contexts, specifically in frontend displays and server-side amount calculations.

Reference: https://github.com/Uniswap/v3-core/issues/573

Recommendation

Ensure the underflows have been properly handled and managed outside the tick and position contexts.

Alleviation

[Cetus Team, 04/18/2025]: The underflows are intentionally handled and managed outside the tick and position contexts, including in frontend displays and server-side calculations.

CEA-17 | Potential Inconsistent Liquidity After The Recovery

Category	Severity	Location	Status
Coding Issue	Informational	sources/position.move (cetus_update): 604~609	Resolved

Description

When adding liquidity to the pool, there are two data structure to store the liquidity value:

- `PositionManager.positions`: `PositionManager` is a shared object.
- `Position`: `Position` is an NFT, owned by the user.

In the emergency recovery process, the `apply_liquidity_cut` function adjusts liquidity solely within the internal `PositionInfo` struct to reflect proportional reductions. This design choice avoids immediate updates to the user-facing Position NFT's liquidity field due to the Sui object ownership, however, potentially leading to temporary inaccuracies in NFT-based queries.

To ensure accuracy, we recommend querying liquidity from the PositionInfo struct rather than the NFT for the `liquidity` function, particularly after recovery operations.

Recommendation

We would like to check with the team if the scenario has been considered.

Alleviation

[Cetus Team, 07/29/2025]: This behavior is **intentional** under the current design.

In the case of liquidity recovery or position removal (e.g., via `emergency_remove_malicious_position`), the liquidity recorded in the `Position` and the actual `PositionInfo` may temporarily become inconsistent. This is expected and accounted for in our protocol logic.

We are currently **upgrading our other contracts** to consistently read the active liquidity **directly from ** `PositionInfo`, rather than relying on the `Position` object. Our frontend and SDK have also been updated accordingly to reflect this change.

Additionally, as soon as the affected position performs **any liquidity operation** (e.g., add/remove), the internal state will be reconciled, and the liquidity values will become consistent again.

[CertiK, 07/31/2025]: The finding is marked as resolved since the following function has been set to deprecated with commit 0df3adad7d3e41c24e64e418d38173c48221fe32:

- `apply_liquidity_cut()`
- `governance_fund_injection()`
- `governance_fund_withdrawal()`

- `emergency_remove_malicious_position()`
- `emergency_restore_pool_state()`

CEA-18 | Discussion On Reward And Position Update

Category	Severity	Location	Status
Logical Issue	Informational	sources/pool.move (cetus_update): 1801~1802	Resolved

Description

The function `emergency_remove_malicious_position` calls `position::remove_position_info_for_restore()` to directly delete the malicious position from the `PositionManager`. This causes any rewards/fees/points accrued by the position to remain locked within the protocol (since no `position::decrease_liquidity` compared with `pool::remove_liquidity`), without being redistributed or burned.

Recommendation

The auditing team would like to confirm with the team whether this behavior is intentional.

Alleviation

[Cetus Team, 07/29/2025]: In practice, **the malicious positions have accumulated little to no rewards or fees**, and we have already **verified all position data and pool balances off-chain** to ensure consistency.

Any fees or rewards previously associated with the attacker's positions are **considered forfeited and will not be redistributed or claimed**.

[CertiK, 07/31/2025]: The finding is marked as resolved since the following function has been set to deprecated with commit 0df3adad7d3e41c24e64e418d38173c48221fe32:

- `apply_liquidity_cut()`
- `governance_fund_injection()`
- `governance_fund_withdrawal()`
- `emergency_remove_malicious_position()`
- `emergency_restore_pool_state()`

CEA-19 | Potential Blocker In Emergency Workflow

Category	Severity	Location	Status
Volatile Code	Informational	sources/pool.move (cetus_update): 1601~1603	Resolved

Description

The system's emergency mode can be activated in two ways:

1. Calling `config.move`'s `update_package_version` function to set the package to an "emergency pause" or "emergency restore" version.
2. Calling `config.move`'s `emergency_pause` function, which sets the version to "emergency pause."

Once the package is in an emergency state, the `checked_package_version` guard will block standard functions that are not designed for recovery. This includes `pool` 's `pause` function.

This behavior creates a strict operational dependency because recovery functions, such as `init_position_snapshot` , require the pool to already be in a paused state (`assert!(pool.is_pause, EPoolNotPaused);`).

Therefore, administrators are forced to follow a specific sequence: they must **pause** the pool(s) *before* activating the emergency mode. Triggering the emergency mode prematurely will make it impossible to pause the pools, thereby blocking essential recovery functions and hindering the entire process.

Recommendation

We would like to check with the team if this is intended.

Alleviation

[Cetus Team, 07/29/2025]: This behavior is **intentional** and aligned with our internal recovery procedure.

In practice, **all affected pools are paused before triggering emergency mode**. After an incident, the pools involved are typically paused immediately, so the condition `pool.is_pause == true` will already be met before any recovery functions (like `init_position_snapshot`) are called.

Additionally, even if the emergency mode has already been activated via `emergency_pause` , administrators can still proceed by using a PTB (Programmable Transaction Block) to:

1. Update the `package_version` back to a non-emergency state temporarily,
2. Call `pause()` on all required pools,
3. Then re-activate the emergency version to proceed with recovery.

This controlled sequence gives us the flexibility to manage recovery steps safely and does not block any necessary operations.

[CertiK, 07/31/2025]: The finding is marked as resolved since the following function has been set to deprecated with commit 0df3adad7d3e41c24e64e418d38173c48221fe32:

- `apply_liquidity_cut()`
- `governance_fund_injection()`
- `governance_fund_withdrawal()`
- `emergency_remove_malicious_position()`
- `emergency_restore_pool_state()`

CEA-20 | Discussion On `emergency_restore_pool_state`

Category	Severity	Location	Status
Logical Issue	Informational	sources/pool.move (cetus_update): 1843	Resolved

Description

In `pool.move`, the `emergency_restore_pool_state` is designed to restore the pool's current price and liquidity to a given state. However, the pool's token balances are not updated, meaning its actual balances and the pool states do not match.

While there is a `governance_fund_injection` function to update the pool's token balances, it does not enforce that the new token balances match the pool states.

Additionally, the `rewarder` is settled at the current timestamp **before** the price and liquidity are stored, meaning the reward and point growth will be distributed to the liquidity providers at the abnormal price with incorrect liquidity.

Recommendation

The auditing team would like to confirm with the Cetus team whether this behavior is intentional.

Alleviation

[Cetus Team, 07/29/2025]: The separation between `emergency_restore_pool_state` and `governance_fund_injection` is intentional. This design supports flexibility during recovery operations, there may be a brief intermediate period where the pool's internal state (e.g., price and liquidity) does not yet match its actual token balances. However, **the system remains closed to public interaction during this process**, and no user operations (trading, liquidity, etc.) are allowed until the full recovery is completed. As such, this temporary mismatch does not pose a security or consistency issue in practice.

In addition, **before initiating recovery**, we explicitly set the reward emission speed of the affected pool to zero. This ensures that **no new rewards are accumulated** during the abnormal state period, and no incorrect reward growth is distributed to LPs.

[CertiK, 07/31/2025]: The finding is marked as resolved since the following function has been set to deprecated with commit 0df3adad7d3e41c24e64e418d38173c48221fe32:

- `apply_liquidity_cut()`
- `governance_fund_injection()`
- `governance_fund_withdrawal()`
- `emergency_remove_malicious_position()`
- `emergency_restore_pool_state()`

CEC-01 Liquidity Rounding Can Strand A Material Portion Of Fixed-Coin Deposits

Category	Severity	Location	Status
Volatile Code	Informational	sources/factory.move (1014-f7f6): 910~911; sources/math/clmm_math.move (1014-f7f6): 413~414; sources/pool.move (1014-f7f6): 692~693, 730~731, 758~759, 2575~2576	Acknowledged

Description

`pool::add_liquidity_fix_coin` forwards the caller's complete fixed-token amount to `clmm_math::get_liquidity_by_amount`. That function rounds the resulting liquidity down, but records the fixed-token payment as the original input amount instead of recomputing the amount represented by the rounded liquidity.

```
if (is_fixed_a) {
    _liquidity = get_liquidity_from_a(
        current_sqrt_price,
        upper_price,
        amount,
        false,
    );
    amount_b = get_delta_b(
        current_sqrt_price,
        lower_price,
        _liquidity,
        true,
    );
    amount_a = amount;
} else {
    _liquidity = get_liquidity_from_b(
        lower_price,
        current_sqrt_price,
        amount,
        false,
    );
    amount_a = get_delta_a(
        current_sqrt_price,
        upper_price,
        _liquidity,
        true,
    );
    amount_b = amount;
}
```

`add_liquidity_internal` credits the position only with the rounded-down `delta_liquidity`, while the receipt retains the complete fixed-token amount.

```
assert!(delta_liquidity > 0, ELiquidityIsZero);

let after_liquidity = position::increase_liquidity(
    &mut pool.position_manager,
    position_nft,
    delta_liquidity,
    fee_growth_inside_a,
    fee_growth_inside_b,
    points_growth_inside,
    rewards_growth_inside,
);

AddLiquidityReceipt<CoinTypeA, CoinTypeB> {
    pool_id: object::id(pool),
    amount_a,
    amount_b,
}
```

The receipt must then be repaid using exactly the recorded amounts. The complete fixed-token input is therefore transferred into the pool even when part of it is not represented by the liquidity credited to the position.

```
assert!(
    balance::value<CoinTypeA>(&balance_a) == amount_a,
    EAmountIncorrect,
);
assert!(
    balance::value<CoinTypeB>(&balance_b) == amount_b,
    EAmountIncorrect,
);

balance::join<CoinTypeA>(&mut pool.coin_a, balance_a);
balance::join<CoinTypeB>(&mut pool.coin_b, balance_b);
```

When the position is removed, the returned principal is calculated exclusively from its credited liquidity. The withdrawal calculation does not retain or use the original fixed-token payment.

```
let (amount_a, amount_b) = clmm_math::get_amount_by_liquidity(
    tick_lower,
    tick_upper,
    pool.current_tick_index,
    pool.current_sqrt_price,
    delta_liquidity,
    false,
);
```

Consequently, the portion of the fixed-token payment that was discarded when liquidity was rounded down remains in the pool balance without a corresponding claim for the depositor.

The issue is most significant when the current price and position range make one integer liquidity unit correspond to a large number of fixed-token base units. If the supplied amount produces only one or a few liquidity units, an amount close to the next liquidity boundary can be charged even though the next unit is not minted.

For example, with tick spacing `200`, a pool price just above the supported minimum, range `[-443600, 0]`, and a fixed input of `8,000,000,000` units of token A, the calculation mints one liquidity unit. Removing that liquidity at the same price returns approximately `4,287,163,120` units of A, leaving `3,712,836,880` units, or approximately `46.41%` of the supplied amount, outside the position's withdrawal claim.

The normal difference between deposit-side rounding up and withdrawal-side rounding down accounts for at most one raw unit in this example. Essentially the entire remaining discrepancy results from retaining the original fixed input after liquidity has been rounded down.

The excess is not associated with the depositor's position and cannot be recovered through an ordinary same-state liquidity removal. It remains in the pool balance and may subsequently participate in pool operations without creating a corresponding entitlement for the original depositor.

The discrepancy is bounded by approximately the fixed-token cost of one liquidity unit. Its relative impact decreases as the amount of minted liquidity increases, but can approach a substantial fraction of the deposit when only one liquidity unit is minted. Its economic value depends on the affected token's decimals and market value.

`factory::create_pool_v3_` propagates the same behavior when initially funding a pool. It limits the required non-fixed token but does not recompute the fixed-token payment after liquidity has been rounded down.

Recommendation

Recall from the validation on router, we would like to check with the team if the scenario has been considered, as well other operation like minimum output(slippage defense) for swap.

Alleviation

[Cetus, 10/15/2026]: We acknowledge the rounding discrepancy and agree that the current fixed-amount path can charge more than the rounded liquidity requires. Existing maximum-input checks do not prevent this.

In the extreme example, approximately 46% of token A is outside the position's withdrawal claim. At the stated pool price, this corresponds to approximately 2 * 10^(-10) raw units of token B. This is a theoretical spot-price valuation, not an executable quote or independent evidence of market value.

To assess practical exposure, we evaluated a production aggregator snapshot covering 3,907 Cetus CLMM pools. At the snapshot prices, the calculated per-addition excess-payment upper bound across all valid position ranges was below $0.001 for every fixed-token side with an available reference price$; $the highest was approximately$0.00081. This assessment does not establish historical cumulative losses or cover unpriced assets, unlisted pools, or future price conditions.

Changing the fixed-amount semantics from an exact payment to a maximum budget would require compatibility review and updates to integrations that split and expect to fully consume an input Coin. Refunding remainders could also create

additional dust Coin objects unless integrations are updated to merge or reuse them.

Given the small absolute impact demonstrated within the assessed scope and these compatibility costs, we currently prefer to retain the existing behavior and explicitly accept the residual risk. We would welcome treating this as a Discussion item or considering a low-risk classification, while documenting the limitations above.

OPTIMIZATIONS | CETUS - CLMM

ID	Title	Category	Severity	Status
CEA-11	Comment Error	Code Optimization	Optimization	Resolved
CEA-16	Unused Remove Function In Position Snapshot	Coding Style	Optimization	Resolved

CEA-11 | Comment Error

Category	Severity	Location	Status
Code Optimization	Optimization	sources/tick.move (cetus_clmm): 620	Resolved

Description

The following typo in the comment, which may mislead developers, should be corrected:

`[-4423636, 443636]` should be `[-443636, 443636]`.

Recommendation

Recommend update the potential typos inside the comment to improve code readability.

Alleviation

[Cetus Team, 04/16/2025]: The team resolved this issue in the commit d00f3268b50ac8acc0c8675feddf5041a8cbb4be.

CEA-16 | Unused Remove Function In Position Snapshot

Category	Severity	Location	Status
Coding Style	Optimization	sources/position_snapshot.move (cetus_update): 202~206	Resolved

Description

The `remove` function in `position_snapshot.move` is designed to remove a position ID from the `PositionLiquiditySnapshot` struct. However, despite its `public(package)` visibility, it is not utilized anywhere in the current production codebase except the tests.

Recommendation

We would like to check with the team if this is intended and ensure no logic is missing.

Alleviation

[Cetus Team, 07/29/2025]: Yes, this is intended.

The `remove` function was originally written as a **fallback mechanism** during the recovery process, in case there was a need to manually remove a position from the snapshot due to data inconsistencies.

However, throughout the entire recovery workflow, this function was **never needed**, and based on our current architecture, **it will not be used going forward** either.

APPENDIX | CETUS - CLMM

Audit Scope

CetusProtocol/cetus-clmm-sui
sources/pool.move
sources/factory.move
sources/config.move
sources/rewarder.move
sources/partner.move
sources/acl.move
sources/math/clmm_math.move
sources/math/tick_math.move
sources/position.move
sources/tick.move
sources/pool_creator.move
sources/utils.move

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Access Control	Access Control findings are about security vulnerabilities that make protected assets unsafe.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.

Categories	Description
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Code Optimization	No description available.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.